

Documentação do Software

Nome do Software: App Scholar

Nome do(a) Aluno(a): Carlos Eduardo da Silva Magalhães

1. Introdução

1.1. Objetivo do Projeto

O projeto tem como objetivo desenvolver um Sistema de Gestão Escolar multiplataforma focado na automatização e facilitação de processos acadêmicos diários. A aplicação visa otimizar a comunicação institucional e centralizar o gerenciamento de registros e históricos de alunos e professores, reduzindo a burocracia de secretarias acadêmicas por meio de um ambiente digital ágil e intuitivo.

1.2. Escopo do Sistema

O sistema mobile é um aplicativo desenvolvido para a plataforma Android utilizando arquitetura híbrida. O seu propósito principal consiste em permitir a autenticação de usuários, a realização de cadastros complexos (com validações de CEP e buscas dinâmicas de estados e cidades via APIs do IBGE), o gerenciamento de dados de turmas, cursos e semestres, além de um sistema ativo de recuperação de credenciais e notificações transacionais de boas-vindas integradas a serviços em nuvem.

1.3. Público-Alvo

Alunos de graduação em tecnologia que utilizam dispositivos móveis para consultar seu boletim acadêmico, **professores** cadastrados na instituição que têm seus dados gerenciados pelo sistema, **coordenadores** e **administradores** institucionais responsáveis pelo cadastro de disciplinas e gerenciamento acadêmico, **desenvolvedores** e **analistas de sistemas** envolvidos na manutenção do backend Node.js e da API REST, e **testadores** responsáveis pela validação das funcionalidades do aplicativo mobile.

1.4. Siglas e/ou Abreviações

- API: Interface de Programação de Aplicações (Application Programming Interface).
- CRUD: Criação, Leitura, Atualização e Exclusão (Create, Read, Update, Delete).
- UI/UX: Interface do Usuário / Experiência do Usuário.
- APK: Pacote de Aplicativo Android (Android Package Kit).
- EAS: Expo Application Services (Serviço de compilação em nuvem da Expo).
- DER: Diagrama de Entidade-Relacionamento.

2. Visão Geral do Sistema

2.1. Funcionalidades Principais

- Autenticação e Controle de Sessão: Login seguro e validação de status de usuários ativos.
- Cadastro Automatizado de Alunos: Módulo integrado com consumo assíncrono da API de CEP e preenchimento de combos de localidade baseados no IBGE.

- Módulo de Professores: Cadastro e edição estruturada contendo titulação, área de atuação e histórico de tempo de docência.
- Comunicação Automatizada: Disparo de e-mails transacionais (boas-vindas com senhas padrão e tokens de segurança para redefinição de acessos).
- Navegação Fluida: Transição dinâmica entre telas com avanço automático por foco de teclado.

2.2. Plataformas Suportadas

- Android: Compatível a partir da versão 8.0 (Oreo) ou superior, distribuído via compilação em formato APK gerada por serviços EAS.

3. Requisitos do Sistema

3.1. Requisitos Funcionais

- **RF001:** O sistema deve permitir a autenticação de usuários via login e senha, retornando token JWT.
- **RF002:** O sistema deve permitir o cadastro, visualização, edição e exclusão de alunos, com campos de nome, matrícula, curso, email, telefone e endereço completo.
- **RF003:** O sistema deve permitir o cadastro, visualização, edição e exclusão de professores, com campos de nome, titulação, área de atuação, tempo de docência e email.
- **RF004:** O sistema deve permitir o cadastro, visualização, edição e exclusão de disciplinas, com campos de nome, carga horária, professor responsável, curso e semestre.
- **RF005:** O sistema deve permitir o registro e consulta de notas por aluno e disciplina, calculando automaticamente a média e a situação (aprovado/reprovado).
- **RF006:** O sistema deve exibir o boletim acadêmico completo do aluno, contendo disciplinas, notas, médias e situação.
- **RF007:** O sistema deve preencher automaticamente o endereço do aluno a partir do CEP informado, consumindo a API externa ViaCEP.
- **RF008:** O sistema deve carregar dinamicamente a lista de estados e cidades a partir da API externa do IBGE Localidades.

3.2. Requisitos Não Funcionais

- **RNF001:** O sistema deve processar as consultas de autenticação e requisições de banco de dados com tempo de resposta ágil.
- **RNF002:** A interface mobile deve ser responsiva e compatível com dispositivos Android e iOS por meio do React Native com Expo.
- **RNF003:** A comunicação entre o aplicativo mobile e o backend deve ser realizada via HTTPS.
- **RNF004:** O sistema deve utilizar autenticação baseada em token JWT para proteger os endpoints da API.
- **RNF005:** O banco de dados PostgreSQL deve manter a integridade referencial entre as tabelas de alunos, professores, disciplinas e notas por meio de chaves estrangeiras.

- **RNF006:** O backend deve ser estruturado seguindo o padrão MVC, com separação entre controllers, routes, models e database.
- **RNF007:** O aplicativo deve ser compatível com os dispositivos móveis mais utilizados no mercado nacional, suportando as versões recentes dos sistemas operacionais Android e iOS.
- **RNF008:** O sistema deve ser capaz de operar com o backend hospedado em ambiente de nuvem, como o Render, sem degradação de funcionalidades.

4. Arquitetura do Sistema

4.1. Tecnologias Utilizadas

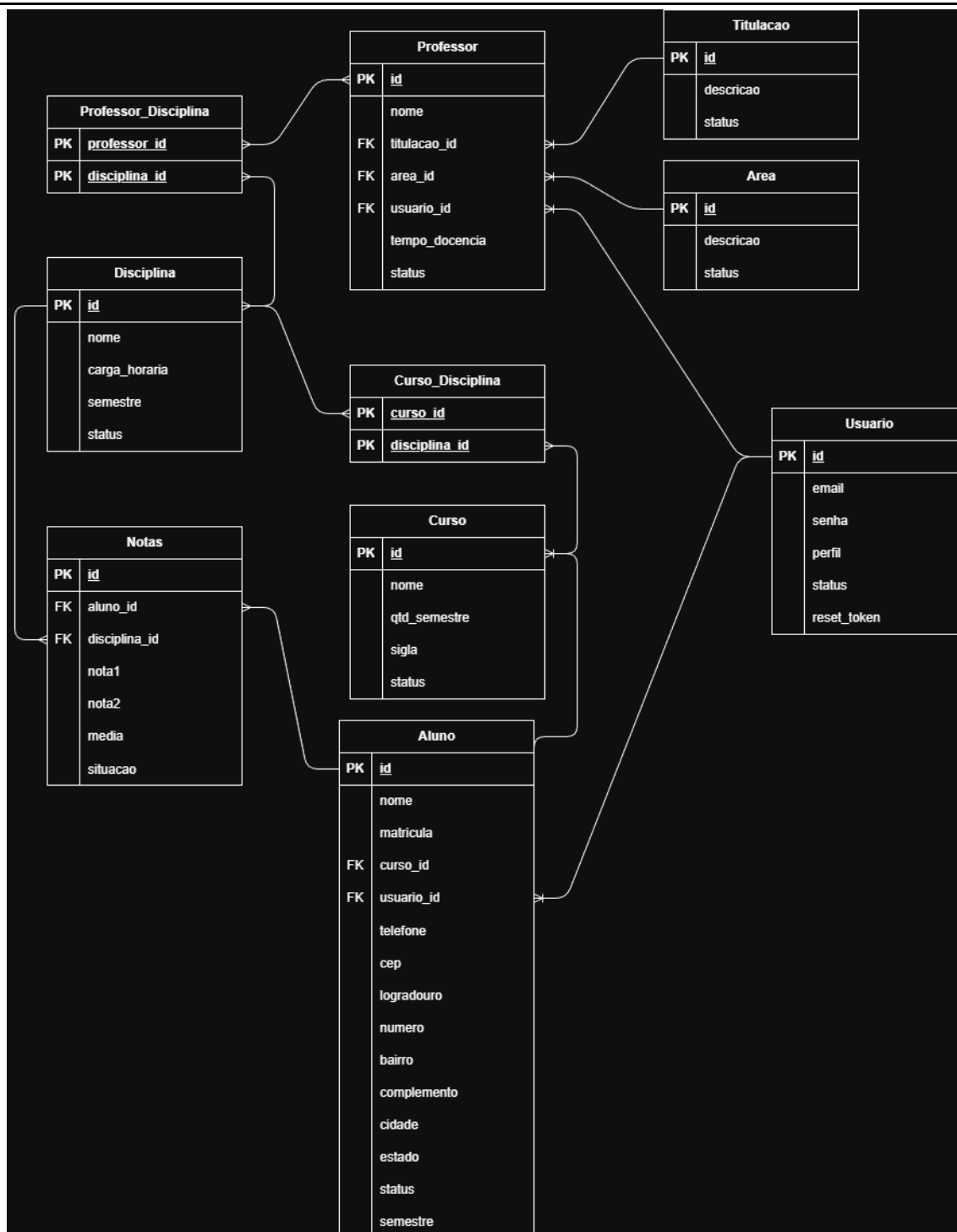
- **Linguagem Principal:** TypeScript (garantindo tipagem estática e segurança em tempo de compilação tanto no Front-end quanto no Back-end).
- **Framework Front-end:** React Native estruturado através do ecossistema Expo e gerenciado via EAS Build.
- **Back-end:** Node.js utilizando o framework Express para disponibilizar rotas e serviços.
- **Persistência e Banco de Dados:** Banco de dados relacional (PostgreSQL/SQL Server) operado por meio do ORM Prisma (Prisma Client).
- **API:** Arquitetura baseada em RESTful com troca de dados estruturada em formato JSON.
- **Serviços Externos de Infraestrutura:**
 - Hospedagem Back-end: Render.
 - Hospedagem Banco de Dados: Supabase.
 - Mensageria/E-mail: Twilio SendGrid Web API.

4.2. Camadas da Arquitetura

- **Apresentação (Mobile App):** Interface construída com componentes customizados em React Native (Inputs inteligentes com referências, botões de ação e Selects baseados em Dropdowns).
- **Serviço / Comunicação:** Camada de requisições HTTP assíncronas encapsuladas em instâncias do Axios (serverApi), responsáveis pela conversação com a API hospedada.
- **Negócio (Back-end API):** Controladores e Middlewares desenvolvidos em Node.js encarregados de processar regras de validação, transações isoladas de banco de dados e gerenciamento de tokens de segurança.
- **Persistência (Database Layer):** Gerenciada pelo Prisma ORM, aplicando migrações estruturadas, execução de scripts de seeds para povoamento inicial de tabelas e isolamento em transações relacionais.

5. Modelagem de Dados

5.1. Diagrama de Entidade-Relacionamento (DER)



5.2. Estrutura de Tabelas

- **Tabela Usuario**

- id: inteiro, chave primária, autoincrementado
- email: texto, único
- senha: texto criptografado
- perfil: enumerado (ALUNO, PROFESSOR, ADMIN)
- status: enumerado (ATIVO, INATIVO), padrão ATIVO

- reset_token: texto, único, opcional

- **Tabela Aluno**

- id: inteiro, chave primária, autoincrementado
- nome: texto
- matricula: texto, único
- curso_id: inteiro, chave estrangeira referenciando Curso
- usuario_id: inteiro, único, chave estrangeira referenciando Usuario
- telefone: texto, opcional
- cep: texto
- logradouro: texto, opcional
- numero: texto, opcional
- bairro: texto, opcional
- cidade: texto, opcional
- estado: texto, opcional
- complemento: texto, opcional
- semestre: inteiro, padrão 1

- status: enumerado (ATIVO, INATIVO), padrão ATIVO

- **Tabela Professor**

- id: inteiro, chave primária, autoincrementado
- nome: texto
- titulacao_id: inteiro, chave estrangeira referenciando Titulacao
- area_id: inteiro, chave estrangeira referenciando Area
- usuario_id: inteiro, único, chave estrangeira referenciando Usuario
- tempo_docencia: inteiro, opcional
- status: enumerado (ATIVO, INATIVO), padrão ATIVO

- **Tabela Disciplina**

- id: inteiro, chave primária, autoincrementado
- nome: texto
- carga_horaria: inteiro

- semestre: inteiro
- status: enumerado (ATIVO, INATIVO), padrão ATIVO
- **Tabela Notas**
 - id: inteiro, chave primária, autoincrementado
 - aluno_id: inteiro, chave estrangeira referenciando Aluno
 - disciplina_id: inteiro, chave estrangeira referenciando Disciplina
 - nota1: decimal (5,2), opcional
 - nota2: decimal (5,2), opcional
 - media: decimal (5,2), opcional
 - situacao: texto, opcional
- **Tabela Curso**
 - id: inteiro, chave primária, autoincrementado
 - nome: texto
 - sigla: texto, opcional
 - qtd_semestre: inteiro, opcional
 - status: enumerado (ATIVO, INATIVO), padrão ATIVO
- **Tabela Titulacao**
 - id: inteiro, chave primária, autoincrementado
 - descricao: texto
 - status: enumerado (ATIVO, INATIVO), padrão ATIVO
- **Tabela Area**
 - id: inteiro, chave primária, autoincrementado
 - descricao: texto
 - status: enumerado (ATIVO, INATIVO), padrão ATIVO
- **Tabela Professor_Disciplina (tabela de associação)**
 - professor_id: inteiro, chave estrangeira referenciando Professor
 - disciplina_id: inteiro, chave estrangeira referenciando Disciplina
 - chave primária composta: (professor_id, disciplina_id)
- **Tabela Curso_Disciplina (tabela de associação)**

- curso_id: inteiro, chave estrangeira referenciando Curso
- disciplina_id: inteiro, chave estrangeira referenciando Disciplina
- chave primária composta: (curso_id, disciplina_id)

6. Fluxos de Navegação

6.1. Telas e Componentes

- Tela de Login.
- Tela de Recuperação de Senha (ForgotPassword).
- Tela Inicial (Dashboard).
- Telas de CRUD (Cadastro/Consulta/Edição/Exclusão).
 - Tela de Formulário Aluno.
 - Tela de Formulário Professor.
 - Tela de Formulário Disciplinas.
 - Tela de Formulário Notas.

6.2. Diagrama de Navegação

1. Autenticação

- a. Login → (credenciais válidas) → Dashboard
- b. Login → Esqueci minha senha → Redefinir senha → Login

2. Dashboard: Após o login, o usuário acessa o Dashboard com os menus disponíveis conforme seu perfil:

- a. ADMIN: visualiza Alunos, Professores, Disciplinas e Boletim
- b. PROFESSOR: visualiza Alunos, Disciplinas e Boletim
- c. ALUNO: visualiza apenas Boletim

O botão de logout encerra a sessão e retorna para a tela de Login.

3. Alunos: Dashboard → Lista de Alunos → Cadastro/Edição de Aluno → Lista de Alunos

4. Professores: Dashboard → Lista de Professores → Cadastro/Edição de Professor → Lista de Professores

5. Disciplinas: Dashboard → Lista de Disciplinas → Cadastro/Edição de Disciplina → Lista de Disciplinas

6. Boletim: Dashboard → Lista de Boletim → Lançamento/Edição de Nota → Lista de Boletim

Em todos os fluxos de CRUD, a tela de formulário adapta seu título dinamicamente para "Cadastro de X" (novo registro) ou "Editar X" (registro existente), e ao confirmar a operação o usuário retorna para a listagem correspondente.

7. Segurança

7.1. Autenticação e Autorização

- O sistema utiliza JWT (JSON Web Token) para autenticação, gerado no momento do login e enviado no cabeçalho das requisições à API para validar o acesso aos endpoints protegidos.
- As senhas dos usuários são armazenadas de forma criptografada no banco de dados utilizando a biblioteca bcrypt, garantindo que nenhuma senha seja persistida em texto puro.
- O controle de autorização é feito por perfil de usuário (ADMIN, PROFESSOR, ALUNO), restringindo o acesso às telas e funcionalidades conforme o nível de permissão de cada perfil.
- O sistema também suporta fluxo de redefinição de senha via token único (reset_token), armazenado de forma segura na tabela de usuários e invalidado após o uso.

7.2. Armazenamento Seguro

- No lado do aplicativo mobile, o token JWT é armazenado utilizando AsyncStorage, isolando as informações de sessão do restante do armazenamento do dispositivo.
- A comunicação entre o aplicativo e o backend é realizada exclusivamente via HTTPS, garantindo a criptografia dos dados em trânsito.
- No backend, o uso do Prisma Client como ORM previne ataques de SQL Injection, pois todas as queries são parametrizadas automaticamente, sem concatenação direta de strings SQL. A validação e o tratamento de dados de entrada nos controllers reduzem a superfície de exposição a entradas maliciosas.

8. Testes

8.1. Tipos de Testes Aplicados

- **Testes de Caixa Preta / Funcionais:** Validação manual do fluxo completo de ponta a ponta (cadastros, edições que não sobrescrevem campos históricos e comportamentos do teclado).
- **Testes de Integração de Infraestrutura:** Validação da camada de rede, atestando o funcionamento do envio de payloads via HTTPS para a API do SendGrid a partir de contêineres dinâmicos do Render.

9. Implantação

9.1. Ambientes

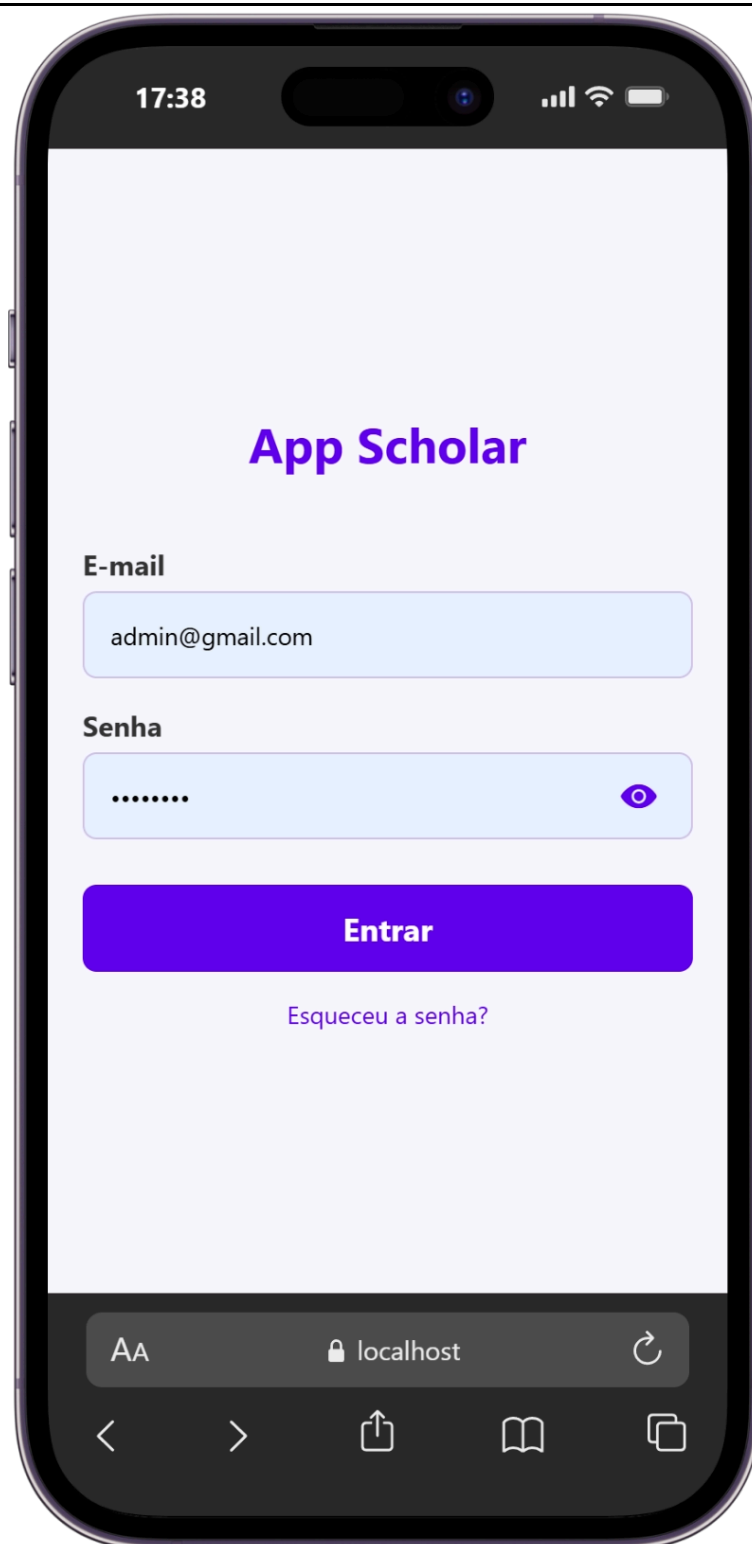
- **Ambiente de Desenvolvimento:** Executado localmente na máquina do desenvolvedor utilizando variáveis locais (.env), simulando chamadas HTTP para localhost.
- **Ambiente de Produção:** Backend implantado na nuvem do Render integrado a bancos de dados remotos e chaves secretas injetadas diretamente nas variáveis de ambiente da plataforma (Environment Variables). O aplicativo mobile é compilado em modo Release (produção) gerando pacotes APK oficiais por meio do EAS CLI.

10. Manutenção e Atualizações

- **Análise de Logs Operacionais:** Utilização do terminal de monitoramento em tempo real fornecido pelo Render para rastrear falhas de conexão, monitorar transações do banco de dados e diagnosticar erros assíncronos em serviços de e-mail.
- **Correções de Layout e UX:** Ajustes dinâmicos em componentes baseados no feedback visual de testes físicos em dispositivos Android (como a aplicação de paddings de escape inferiores no ScrollView para evitar cortes visuais de dropdowns e listas suspensas).

11. Anexos

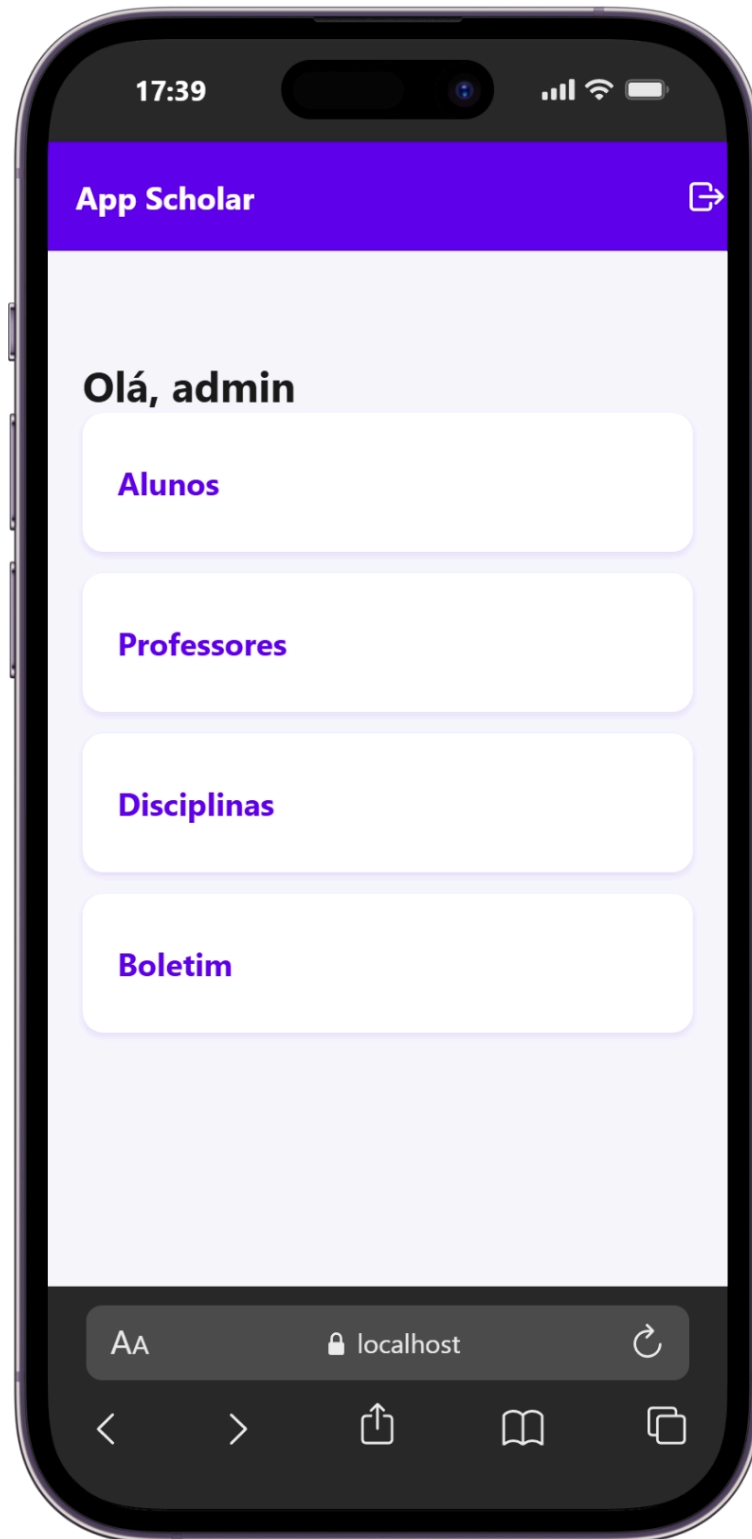
- **Prints das telas**
 - Login



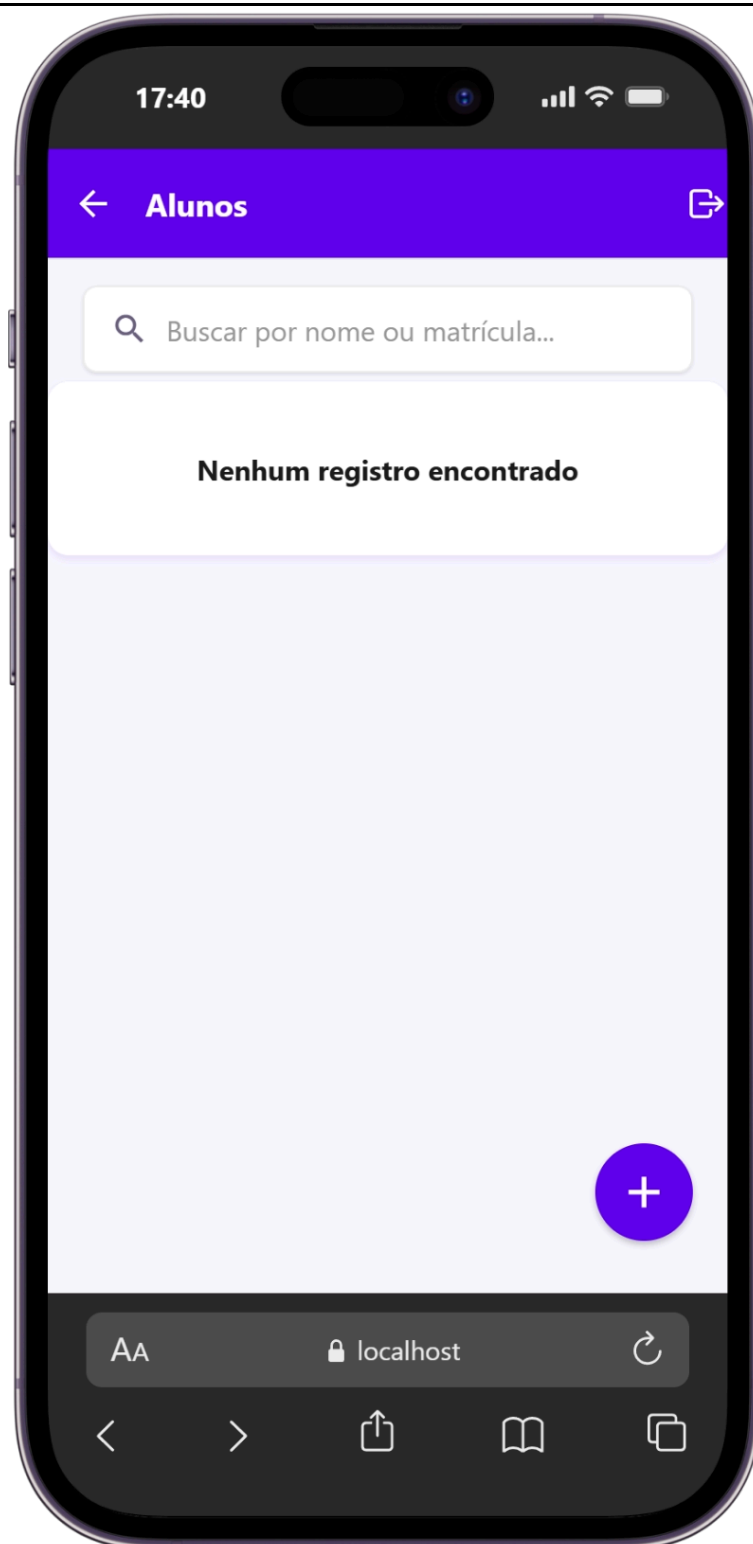
- Redefinir senha



- Dashboard



- Formulário de alunos



17:41

← **Cadastro de Aluno** →

Nome Completo

Digite o nome completo

Matrícula

Digite a matrícula

Curso

Selecione o curso ▼

Semestre

Selecione o semestre ▼

Email

Digite o email

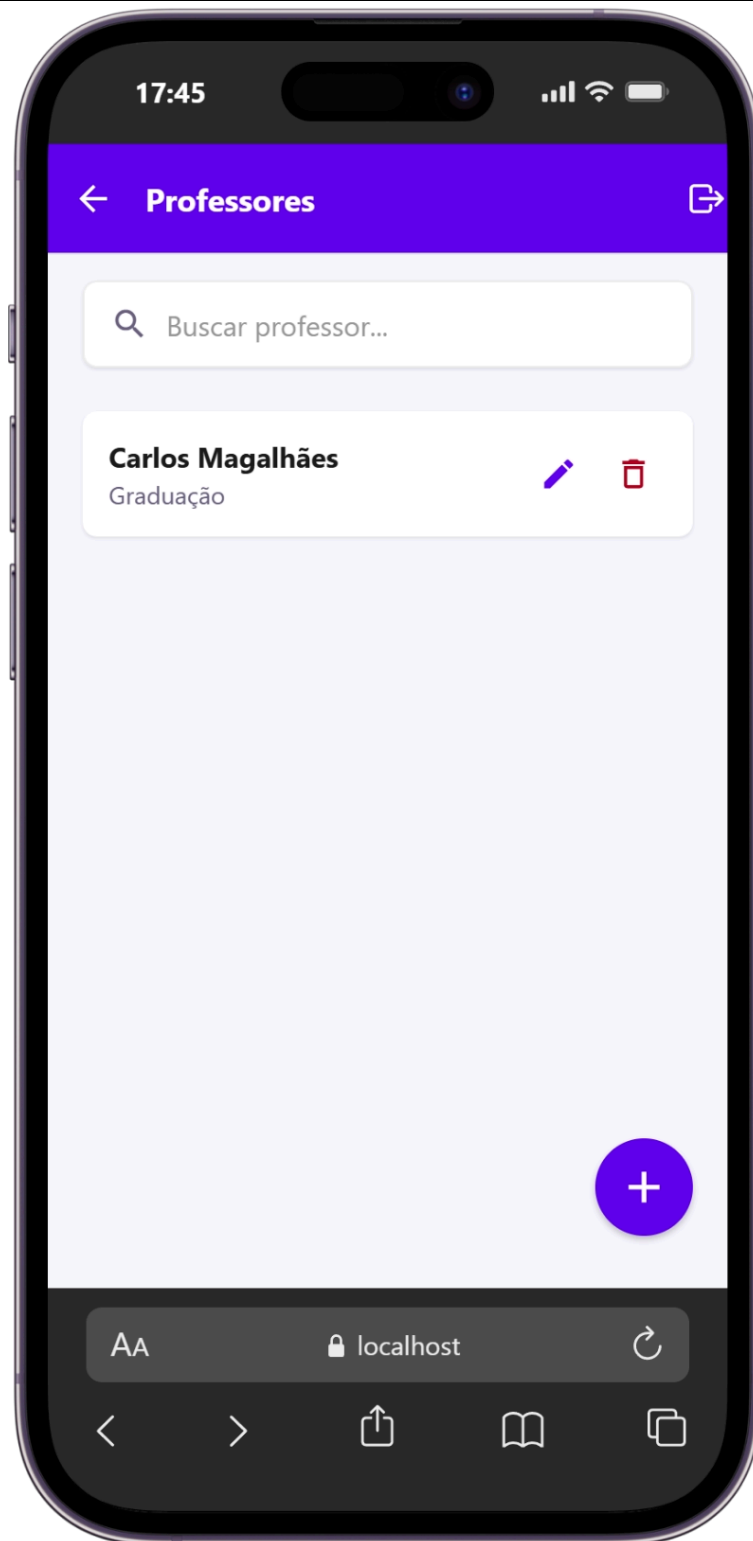
Telefone

Digite o telefone

AA localhost ↺

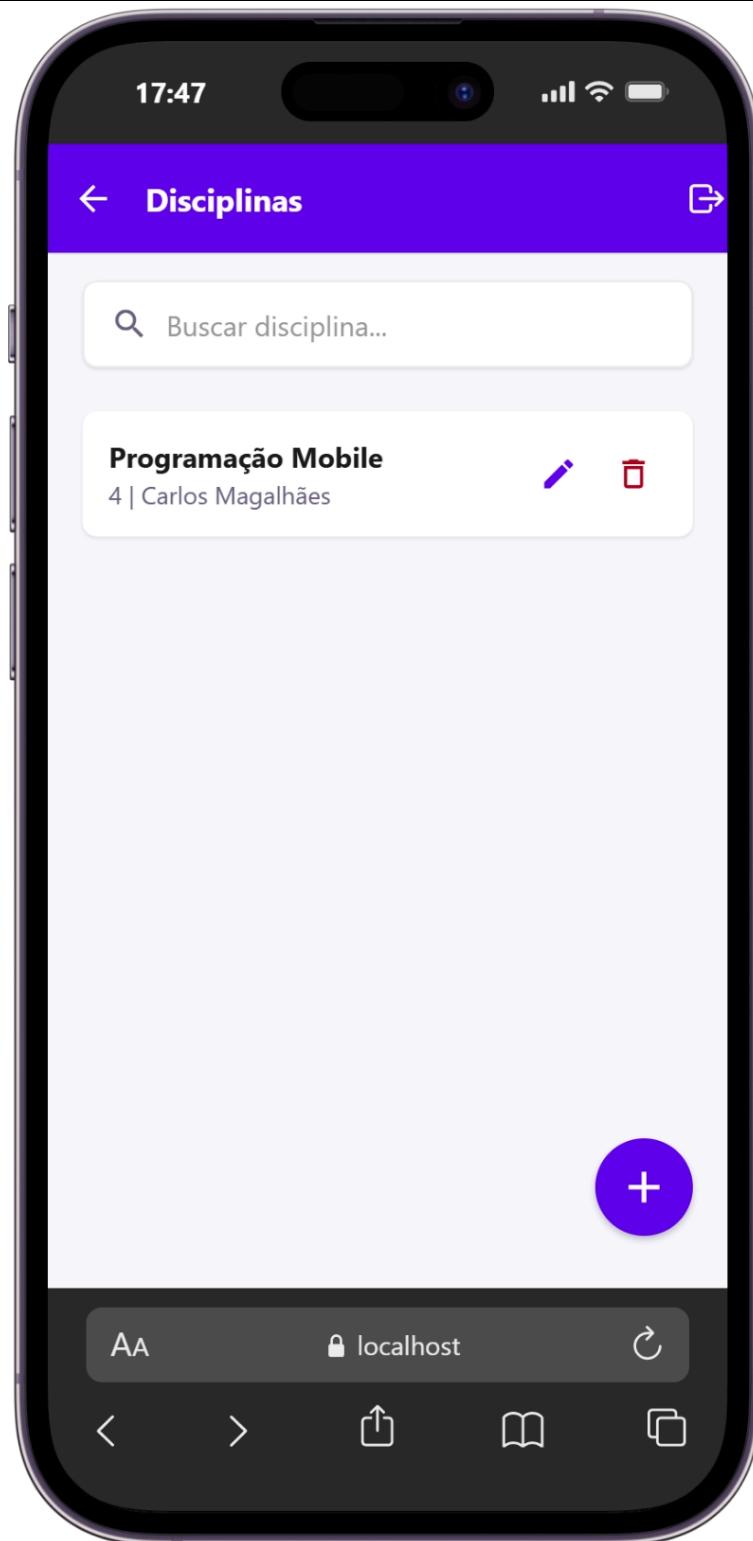
< > ↗ 📖 📄

- **Formulário de professores**



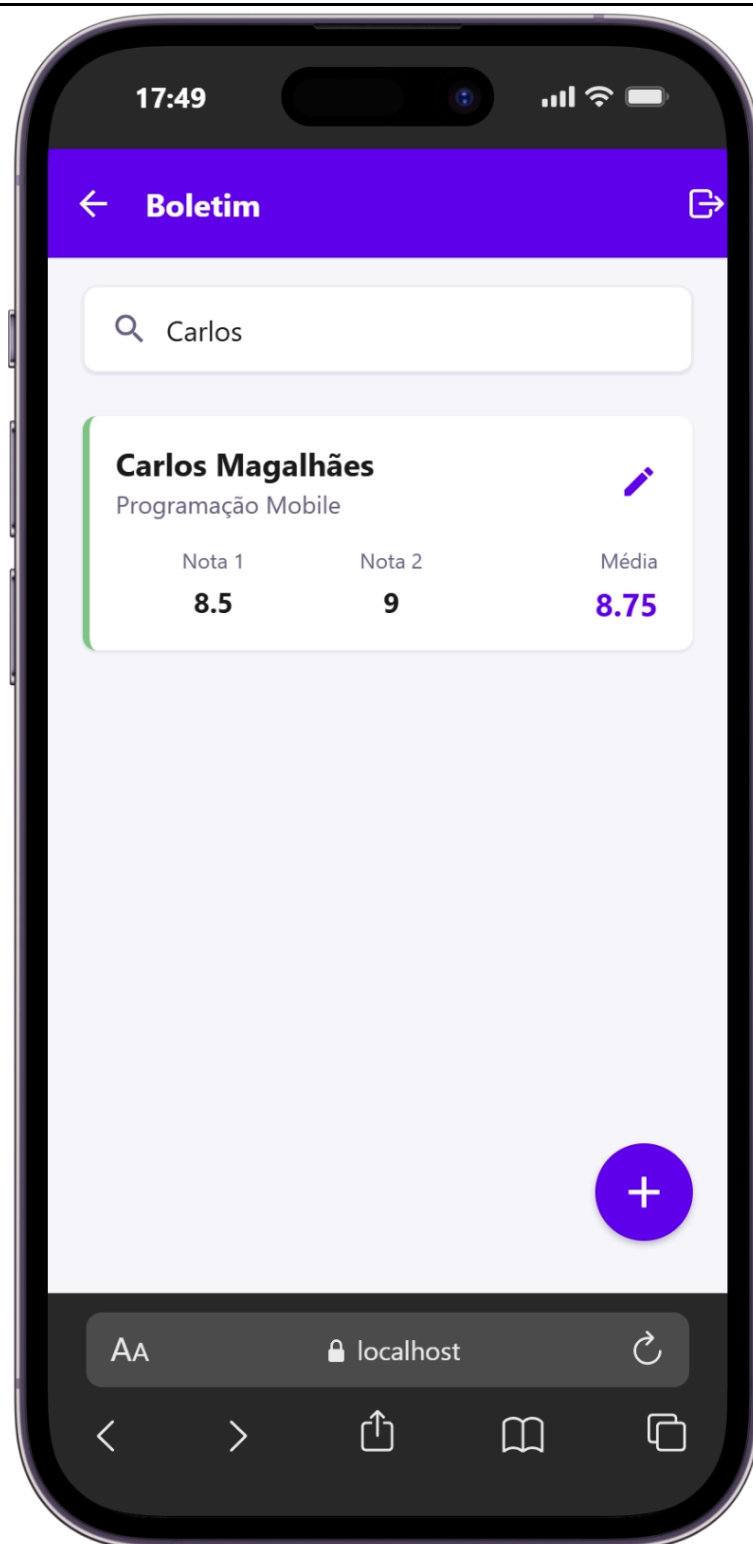
The image shows a smartphone screen displaying a web application interface for editing a professor's profile. The status bar at the top shows the time 17:46, signal strength, Wi-Fi, and battery levels. The app's header is a purple bar with a back arrow, the title 'Editar Professor', and a share icon. The form contains several fields: 'Nome Completo' with the value 'Carlos Magalhães', 'Email' with 'carlos123cadu12355@gmail.com', 'Titulação' with a dropdown menu showing 'Graduação', 'Área de Atuação' with a dropdown menu showing 'Tecnologia da Informação', and 'Tempo de Docência (anos)' with the value '1'. A large purple 'Confirmar' button is at the bottom of the form. The bottom of the screen shows a browser's address bar with 'localhost' and a standard mobile OS navigation bar with icons for back, forward, share, book, and tabs.

- Formulário de disciplinas



The image shows a smartphone screen displaying a web application interface for editing a discipline. The status bar at the top shows the time 17:47, signal strength, Wi-Fi, and battery levels. The app's header is purple with a back arrow, the title 'Editar Disciplina', and a share icon. The form contains several sections: 'Nome da Disciplina' with a text input field containing 'Programação Mobile'; 'Carga Horária (horas)' with a text input field containing '80'; 'Curso' with a dropdown menu showing 'Desenvolvimento de Sistemas' and 'Multiplataforma'; 'Semestre' with a dropdown menu showing '4º Semestre'; and 'Professores' with a text input field containing 'Carlos Magalhães' and a red 'X' button to remove it. Below the list of professors is a dashed border box with a purple '+ Adicionar professores' button. At the bottom of the screen, there is a browser address bar showing 'AA' and 'localhost' with a refresh icon, and a navigation bar with icons for back, forward, share, book, and tabs.

- Formulário de notas



17:50

← Editar Nota →

Disciplina

Programação Mobile - 4º Sem

Aluno

Carlos Magalhães (123)

Nota 1

8.5

Nota 2

9

Confirmar

AA localhost ↺

< > ⬆ ⬇ 📖 📄